QueryAPIGate HOW-TO GUIDE 02 · START

# Connect to PostgreSQL, MySQL, SQLite, DuckDB, ClickHouse or H2

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=02-connect-a-database

---

**Time:** 5-15 minutes depending on the database. **You'll end up with:** a real, active connection you can query through QueryAPIGate - the admin UI's schema browser will list its tables, and the API Designer will run real SQL against it.

Every connection - regardless of database type - is one JSON object with the same handful of fields: `db`, `host`, `port`, `database`, `user`, `password`, `active`. This guide shows the exact shape for each of the six built-in SQL dialects, and both ways to add one: the admin UI, or the API directly.

What each type supports - read-only enforcement, time limits, `allowed_tables`, schema browsing, streaming - is in the support matrix. PostgreSQL, MySQL, SQLite, DuckDB and ClickHouse are tier 1: every feature, tested against a real server in CI. H2 is experimental.

For anything not listed here - Oracle, SQL Server, Snowflake, DB2, or any other JDBC-compliant database - see Connect to anything else via generic JDBC (experimental); the mechanics below (test, activate, query) are identical once a `jdbc` connection exists. For **Trino** - one connection federating many catalogs - there's a native, governed type with `allowed_tables`, masking and a schema browser: Connect to Trino (experimental).

## Two ways to add a connection

**Admin UI**: open **http://127.0.0.1:5000/console**, go to **Connections** (under **Data** in the sidebar), click **New connection**, fill in the form, **Test** it, then **Save**.

**API directly** (what this guide's examples use, since it's copy-pasteable and identical for every dialect):

```shell
curl -X POST http://127.0.0.1:5000/api/v1/connections -H 'X-API-Key: demo-key' -H 'Content-Type: application/json' -d '{"name": "<a name you choose>", "db": "...", ... }'
```

`POST /api/v1/connections` creates one (a name already taken is a 409); `PATCH /api/v1/connections/<name>` changes some of an existing one's fields and keeps the rest. Test any connection's fields (saved or not) before committing to them:

```shell
curl -X POST http://127.0.0.1:5000/api/v1/connections/test -H 'X-API-Key: demo-key' -H 'Content-Type: application/json' \
  -d '{"db": "...", ...}'
# {"elapsed_ms": 12.4}
```

**A faster starting point**: `queryapigate init` (run once, in an empty `QUERYAPIGATE_HOME`) seeds an *inactive* template connection for every supported type - `example-sqlite`, `example-postgres`, `example-mysql`, `example-clickhouse`, `example-h2`, `example-duckdb`. Edit one's real values and flip `"active": true` instead of writing the JSON from

scratch.

## SQLite - no server needed

The simplest case: `database` is a file path (relative paths resolve against `QUERYAPIGATE_HOME`).

```shell
curl -X POST http://127.0.0.1:5000/api/v1/connections -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{"name": "shop", "db": "sqlite", "database": "shop.db", "active": true}'
```

If `shop.db` doesn't exist yet, QueryAPIGate doesn't create it - point this at a real SQLite file, or create an empty one first (`sqlite3 shop.db "CREATE TABLE ..."`, or any SQLite client). Verified for this guide: a real `shop.db` with an `orders` table, connected, tested, and queried successfully.

## PostgreSQL

```shell
curl -X POST http://127.0.0.1:5000/api/v1/connections -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "shop-pg",
  "db": "postgres", "host": "localhost", "port": 5432, "database": "shop",
  "user": "postgres", "password": "${POSTGRES_PASSWORD}", "active": true
}'
```

Needs the driver: `pip install "queryapigate[postgres]"` (already included if you installed `queryapigate[all]` or the published Docker image). `${POSTGRES_PASSWORD}` is a reference to an environment variable, resolved when the connection is actually used - keep the real password out of `queryapigate.db` entirely; see Keep connection passwords out of the store for the alternative (encrypting a literal value instead of using an env reference). Verified for this guide against a real PostgreSQL 16 container: connect, test, query all worked as shown.

## MySQL

```shell
curl -X POST http://127.0.0.1:5000/api/v1/connections -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "shop-mysql",
  "db": "mysql", "host": "localhost", "port": 3306, "database": "shop",
  "user": "root", "password": "${MYSQL_PASSWORD}", "active": true
}'
```

Needs `pip install "queryapigate[mysql]"`. Same `${VAR}` convention for the password. Verified for this guide against a real MySQL 8 container: connect, test, query all worked as shown.

## ClickHouse

```shell
curl -X POST http://127.0.0.1:5000/api/v1/connections -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "events-ch",
  "db": "clickhouse", "host": "localhost", "port": 9000, "database": "default",
  "user": "default", "password": "${CLICKHOUSE_PASSWORD}", "active": true
}'
```

Needs `pip install "queryapigate[clickhouse]"`. Note the port: **9000**, ClickHouse's native protocol port, not 8123 (HTTP) - `clickhouse-driver` speaks the native protocol. ClickHouse also honors backslash escapes inside quoted string literals (like MySQL, unlike Postgres/SQLite/H2) - QueryAPIGate's SQL guard already accounts for this per-dialect, nothing you need to configure.

## DuckDB - no server needed either

An embedded analytical database: `database` is its own `.duckdb` file, which must already exist - like SQLite, QueryAPIGate never creates one, so a mistyped path fails rather than opening an empty database. Create it with `python -c "import duckdb; duckdb.connect('analytics.duckdb')"` (or the DuckDB CLI). It can also read CSV, Parquet and JSON files - local, in a bucket or on the web - straight from SQL, but only the ones its `allowed_paths` lists (guide 41); see DuckDB connections.

```shell
curl -X POST http://127.0.0.1:5000/api/v1/connections -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{"name": "analytics", "db": "duckdb", "database": "analytics.duckdb", "active":
true}'
```

Needs `pip install "queryapigate[duckdb]"`. One difference from SQLite worth knowing: DuckDB has no read-only session to fall back on, so the read-only default rests on QueryAPIGate's SQL check alone.

## H2

> **Experimental** - may change in any minor release, always noted in the changelog (what that means).

H2 is different from the others: QueryAPIGate connects to a *running H2 TCP server*, not an embedded file directly - and needs a JVM (either the `-h2` Docker image variant, or a local Java install plus `pip install "queryapigate[h2]"`).

```shell
curl -X POST http://127.0.0.1:5000/api/v1/connections -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "shop-h2",
  "db": "h2", "host": "localhost", "database": "test",
  "user": "SA", "password": "${H2_PASSWORD}", "active": true
}'
```

This connects to `jdbc:h2:tcp://localhost/~/test` under the hood (add `"port": <n>` if your H2 server isn't on H2's default TCP port, 9092). Two things worth knowing before you rely on this connection: the read-only guarantee rests entirely on the SQL guard, not the database connection itself (`Connection.setReadOnly()` is advisory in JDBC, not enforced), and `allowed_tables` isn't supported - a key restricted to tables is refused on an H2 connection rather than let through. `QUERYAPIGATE_QUERY_TIMEOUT` does apply.

## After connecting: verify and explore

Whichever dialect, the same two checks apply:

```shell
# List every connection and its live usage stats
curl http://127.0.0.1:5000/api/v1/connections -H 'X-API-Key: demo-key'

# Browse its schema - the same introspection the admin UI's schema browser uses
curl http://127.0.0.1:5000/connections/shop/schema -H 'X-API-Key: demo-key'
```

A connection you're not ready to use yet, or want to disable temporarily, doesn't need deleting - set `"active": false` instead; `GET /api/v1/connections/<name>` always masks a plain-text password as `********` either way.

## Next steps

- Turn a query into a REST API against your new connection instead of the built-in example one.
- Keep connection passwords out of the store - encryption, a real alternative to `${VAR}` references.
- Connect to MongoDB - `find()` queries instead of SQL.
- Connect via generic JDBC - Oracle, SQL Server, Snowflake or another JDBC-only database.